Leibniz FH

Wirtschaftsinformatik (B. Sc.)
Modul 20.1 (Vertiefung Entwicklung)
Sommersemester 2026

Anwendung erweiterter Softwareentwicklung

4. OOP (Objekt-orientierte Programmierung)

Dr. Peter Lohmann

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

Leibniz FH

4.1. Konzepte

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

Grundidee der Objektorientierung

- ✓ Objekte als gekapselte Einheiten
  - Zustand über (private) Attribute
  - Verhalten über Methoden
    - „Aufforderung ans Objekt“
- ✓ Motivation
  - mehrere gleichartige Dinge modellieren
  - Daten und Operationen zusammenhalten
    - Durchaus kontrovers: Kann Abhängigkeiten verschleiern

Klassen und Instanzen

- ✓ Klasse
  - Bauplan für Objekte
  - gemeinsame Struktur und gemeinsames Verhalten
- ✓ Instanz
  - konkretes Objekt zur Laufzeit
  - eigener Zustand pro Objekt
- ✓ JavaScript: Prototypen statt Klassen
  - Klonen statt instanziieren

Attribute, Konstruktoren und self

- ✓ Instanzattribute vs Klassenattribute
  - Daten pro Objekt vs globale Eigenschaften
- ✓ Instanzmethoden vs Klassenmethoden / statische Methoden
  - Verhalten pro Objekt vs gekapselte Funktionen
  - Python: Methode = Attribut, das Funktion ist
- ✓ `self`
  - Referenz auf das aktuelle Objekt
  - Dieser Variablenname ist nur Konvention!
- ✓ `__init__(self, ...)`
  - Initialisierung neuer Objekte (Konstruktor)
  - Läuft im Kontext des schon erzeugten Objekts
  - Magic (aka *dunder*) method

Referenzen und Kopien

- ✓ Sharing und Aliasing
  - Variablen speichern (nur) Referenzen auf Objekte!
  - mehrere Namen für dasselbe Objekt
  - automatische Destruktion, wenn keine Referenz mehr übrig
- ✓ Kopieren
  - shallow copy, deep copy

Leibniz FH

4.2. Polymorphie

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

Subtyping

- ✓ Subtypen überall nutzbar, wo Supertyp erwartet wird
  - Python: Keine Garantie!
  - Aber: Wichtiges Design-Prinzip (***L***iskov substitution principle, s. SO***L***ID)
- ✓ gleiche Operation, verschiedene konkrete Objekte
- ✓ annotierter Typ
  - statische Beschreibung
  - Python: Annotation ohnehin bloß „netter Hinweis“
- ✓ Laufzeitobjekt
  - tatsächliches Verhalten

Dynamisches Binden

- ✓ Methodenwahl zur Laufzeit
- ✓ Auflösung über *Objekt/Instanz*, Klasse und Basisklassen
- ✓ Nutzen
  - Erweiterbarkeit
  - weniger Typfallunterscheidungen
- ✓ Gegensatz:
  - Statisches Binden = konkrete Funktion/Implementierung zur Compile-Zeit bekannt

Vererbung und super

- ✓ Subklassen übernehmen Attribute und Methoden
- ✓ Spezialisierung
  - *Extend* (neue Methoden) oder *Override* (neue Implementierungen alter Methoden)
- ✓ `super()`
  - vererbte Initialisierung
  - Zugriff auf Oberklassenverhalten

Mehrfachvererbung und MRO

- ✓ Mehrfachvererbung
  - Klasse erbt von mehreren Basisklassen
  - nützlich für kombinierbares Verhalten
- ✓ MRO = Method Resolution Order
  - legt Suchreihenfolge für Attribute und Methoden fest
  - bestimmt, welche Implementierung gefunden wird
- ✓ `super()` in Hierarchien
  - arbeitet entlang der MRO
  - ermöglicht kooperative Methodenaufrufe

Interfaces

- ✓ Python: *Abstract Base Class (ABC)*
- ✓ unvollständige Oberklassen
- ✓ abstrakte Methoden ohne Implementierung
- ✓ nicht direkt instanziierbar
- ✓ Nutzen
  - klarere Entwürfe
  - standardisierte Spezifikation
  - ***D***ependency Inversion (s. SOLI***D***)

Overriding und Overloading

- ✓ Overriding
  - neue Implementierung geerbter Methoden
- ✓ Overloading
  - signature-basierte Auswahl der Implementierung
  - Python: gibt es nicht
    - stattdessen meist Default-Werte oder flexible Parameter
- ✓ Shadowing
  - gleichnamige Namen überlagern vorhandene Definitionen